



QSOE

Quick and Secure Operating Environment

C Library Reference

What libc gained between QSOE 0.1 and 0.2

Document: QSOE-LCR

Revision: Rev A — Draft

Applies to: QSOE 0.2

Yuri Zaporozhets

September 2026

© 2026 Yuri Zaporozhets.

Licensed under CC BY-ND 4.0 — verbatim redistribution with attribution; no derivatives.

<https://creativecommons.org/licenses/by-nd/4.0/>

Contents

1	About this manual	1
1.1	What is here, and what is not	1
1.2	The four things that are never here	1
1.3	How to read an entry	2
1.4	Applies to	2
2	Processes, execution and sessions	3
2.1	The <code>posix_spawn_file_actions</code> family	3
2.2	<code>wait()</code>	4
2.3	<code>system()</code>	5
2.4	<code>popen()</code> , <code>pclose()</code>	6
2.5	<code>setsid()</code>	6
2.6	<code>tcsetctty()</code>	7
2.7	<code>execve()</code> , <code>execv()</code>	8
3	Pipes and descriptors	9
3.1	<code>pipe()</code> , <code>pipe2()</code>	9
3.2	<code>qsoe_capture()</code>	10
3.3	<code>dup()</code>	10
3.4	<code>__qsoe_close_fds()</code>	11
4	Files, directories and attributes	13
4.1	<code>mkdir()</code>	13
4.2	The <code>chmod</code> and <code>chown</code> families	14
4.3	<code>truncate()</code> , <code>ftruncate()</code>	15
4.4	<code>mkstemp()</code> , <code>mkdtemp()</code>	16
4.5	<code>rename()</code>	16
5	Directory streams	18
5.1	<code>dirfd()</code>	18
5.2	Moving within a directory stream	18
5.3	<code>scandir()</code> , <code>alphasort()</code>	19
6	Formatted and line-oriented input	21
6.1	The <code>scanf</code> family	21
6.2	<code>getline()</code> , <code>getdelim()</code>	22
6.3	<code>rewind()</code> , <code>fgetpos()</code> , <code>fsetpos()</code>	23
7	Numeric conversion, mathematics and random numbers	25
	Note on <code>long double</code>	25
7.1	<code>strtod()</code> , <code>strtof()</code> , <code>strtold()</code> , <code>atof()</code>	25
7.2	Floating-point building blocks	26

7.3	llabs(), imaxabs()	26
7.4	ffs(), ffs(), ffsll()	27
7.5	rand(), srand(), random(), srandom()	27
8	Strings, memory and multibyte characters	29
8.1	strncasecmp()	29
8.2	strcasestr()	30
8.3	memmem()	30
8.4	strndup()	30
8.5	strerror_r()	31
8.6	dirname()	31
8.7	mbrtowc(), wctomb(), mbsinit()	32
9	Time and clocks	34
9.1	mktime(), timegm()	34
9.2	Broken-down time: gmtime, asctime, ctime	35
9.3	difftime()	35
9.4	clock_getres()	35
9.5	usleep()	36
10	Diagnostics	37
10.1	The err and warn family	37
10.2	slog_setlevel(), slog_getlevel()	38
11	System identity and configuration	40
11.1	uname()	40
11.2	gethostname(), sethostname()	41
11.3	qsoe_become_user()	42
11.4	qsoe_sysfiles_reload()	42
12	Device control	44
12.1	devcontrol()	44
13	Shared memory and physical allocation	46
13.1	qsoe_region_publish()	46
13.2	qsoe_map_region()	47
13.3	qsoe_region_revoke()	48
13.4	qsoe_alloc_phys_flags()	49
13.5	qsoe_fdt_region()	50
14	Node descriptors and the net	51
	What a node descriptor is	51
14.1	qsp_strtond()	51
14.2	qsp_ndtostr()	52

Chapter 1

About this manual

1.1 What is here, and what is not

This manual documents the functions the QSOE C library gained *between release 0.1 and release 0.2*. It is a delta, not a complete reference.

That is a deliberate choice and worth stating plainly, because a reader who opens a book called “C Library Reference” reasonably expects to find `printf()` in it. The reason is that most of `libc.so` is musl-derived and behaves exactly as its documentation says it does; restating several hundred standard pages would produce a large manual whose value per page is near zero, and — worse — would bury the handful of functions where QSOE genuinely differs. What is worth writing down is what changed, what is ours, and what behaves differently here than the standard would lead you to expect. That is this book.

For a function that is not here:

- If it is a standard C or POSIX function, it was already present in QSOE 0.1 and behaves as the standard describes. The *Application Porting Guide* (QSOE-APG) lists the parts of POSIX that QSOE deliberately does not implement, and why.
- If it is a QNX-style message-passing call — one of the `ChannelCreate()`, `MsgSend()` or `InterruptAttach()` families — it is described in the *Design & Architecture* guide (QSOE-DSN) and in the *Programming Book* (QSOE-PRG).

1.2 The four things that are never here

QSOE omits four familiar interfaces on purpose rather than by omission, and no amount of searching this manual will turn them up:

fork() There is no `fork()`. A process is created by `posix_spawn()`, which states in one call what `fork()` plus `exec()` states in two — and does not require the kernel to be able to duplicate an address space, which is what copy-on-write exists to make affordable. Chapter 2 covers the file-action interface that replaces the work usually done between the `fork` and the `exec`.

select() `poll()` is the only multiplexer.

brk() Low-level memory comes from the `mmap` family.

ioctl() Device control is `devcontrol()`, which carries the subsystem, the direction and the command as three unsigned values instead of packing them into one signed integer. It is chapter 12, and it is the one substitution in this list that a porter meets immediately.

1.3 How to read an entry

Each entry follows the shape QNX uses, which QSOE keeps because a reader coming from that world should not have to learn a new one:

Synopsis The header to include and the exact declaration.

Arguments Each parameter, in declaration order.

Description What the function does.

Returns The success value and the failure value.

Errors The `errno` values this implementation actually sets — not the full list the standard permits, but the ones our code produces.

Functions that share a page share it because they share behavior: the `scanf()` family differs only in where the input comes from, and documenting them apart would be six copies of one description with one line changed.

Two kinds of remark are set apart from the prose:

QSOE note. A QSOE-specific deviation. If you know the standard behavior of a function, these paragraphs are the ones that will surprise you, and they are boxed so that skimming cannot miss them.

Not implemented. The function exists and links, but does not do the job. Every one of these announces itself on the console when called — a project rule, so that a program which reaches one says so rather than failing quietly somewhere else later.

1.4 Applies to

QSOE 0.2, on both kernels. Everything in this manual is part of the shared `libc.so` and behaves identically on QSOE/N (Skimmer) and QSOE/L (seL4) unless an entry says otherwise; where an entry does say otherwise, the difference is named and explained rather than left for you to find.

Chapter 2

Processes, execution and sessions

QSOE 0.1 could start a process. QSOE 0.2 can start one *with its descriptors already arranged*, wait for it, run a shell command, and put a process at the head of its own session with a controlling terminal. Those four capabilities are what turned a system that could run programs into one that can run a shell with pipelines and job control, and they are the subject of this chapter.

The absence of `fork()` shapes all of it. On a conventional system a caller arranges a child's descriptors by running `dup2()` and `close()` *in the child*, between the fork and the exec. QSOE has no child to run that code in, which is exactly the case POSIX's spawn file actions were designed for: instead of performing the setup, the caller *describes* it, and the spawn applies the description while building the child.

That is not a workaround for a missing primitive; it is the better shape. A caller never disturbs its own descriptor table in order to prepare somebody else's, so there is no save-and-restore dance around a spawn, no window in which the caller's own `stdout` points somewhere surprising, and two children with different descriptors can be prepared at the same time. The whole class of “which end of the pipe did I forget to close” bugs comes from a child inheriting everything by default; here the child's set is exactly what was asked for.

2.1 The `posix_spawn_file_actions` family

Describe the descriptors a spawned child is to be given.

Synopsis

```
#include <spawn.h>

int posix_spawn_file_actions_addopen(
    posix_spawn_file_actions_t *restrict fa,
    int fd, const char *restrict path, int oflag, mode_t mode);

int posix_spawn_file_actions_addclose(
    posix_spawn_file_actions_t *fa, int fd);

int posix_spawn_file_actions_adddup2(
    posix_spawn_file_actions_t *fa, int fd, int newfd);

int posix_spawn_file_actions_destroy(posix_spawn_file_actions_t *fa);
```

Arguments

fa An action list, previously initialized with `posix_spawn_file_actions_init()`.

fd The descriptor the action concerns: the one to open onto, the one to close, or the source of the duplication.

path, oflag, mode

For `addopen`, the arguments an `open()` would have taken. The path is copied into the list, so the caller's buffer need not survive.

newfd

For `adddup2`, the descriptor number `fd` is to appear as in the child.

Description

Each call appends one action to **fa**. When the list is later handed to `posix_spawn()`, the actions are applied *in the order they were added*, against the child's descriptor table, before the child's first instruction runs.

Order matters and is the caller's responsibility. The idiom for a pipeline stage is to add the `adddup2` that puts the pipe end in place first, and the `addclose` of the original descriptor second; reversing them closes the descriptor that the duplication was about to read.

`posix_spawn_file_actions_destroy()` releases the list's storage. A list may be destroyed as soon as the spawn that used it has returned; the child holds no reference to it.

QSOE note. The action list is the *only* way to give a child a descriptor set other than the parent's. With no `fork()` there is no other moment at which a caller could act on the child, so a program ported from a system that manipulated descriptors between `fork` and `exec` must move that code here. There is no partial substitute.

Returns

Zero on success. On failure, the error number itself is returned — these functions do not set `errno`, following POSIX.

Errors

EBADF

A descriptor argument was negative.

ENOMEM

The list could not be grown to hold another action.

See also

`posix_spawn()`, `pipe()` (chapter 3).

2.2 wait()

Wait for any child process to terminate.

Synopsis

```
#include <sys/wait.h>

pid_t wait(int *status);
```

Arguments

status

Where to store the child's exit status, or `NULL` to discard it. The `WIFEXITED()`, `WEXITSTATUS()` and related macros interpret it.

Description

Blocks until any one child of the calling process terminates, and reports which. It is exactly `waitpid(-1, status, 0)`, and is provided because a great deal of existing code calls it by that name.

Returns

The process ID of the child that terminated, or `-1` with `errno` set.

Errors

ECHILD

The calling process has no children to wait for.

EINTR

A signal was caught before any child terminated.

2.3 system()

Run a command through the shell.

Synopsis

```
#include <stdlib.h>

int system(const char *cmd);
```

Arguments

cmd

The command line to run, or `NULL` to ask whether a shell is available at all.

Description

Spawns `/bin/sh` with `-c` and the given command line, waits for it, and reports its exit status. `/bin/sh` is a link to `qsh` in the boot package, so it exists before any filesystem has been mounted — which is what makes `system()` usable from an early-boot program.

Because the command goes to a shell, everything the shell does applies: redirection, pipelines, quoting and variable expansion all work, and so does every hazard of passing unsanitized text to a shell.

Returns

With `cmd` non-NULL: the shell's termination status, in the form `waitpid()` reports, or `-1` if the shell could not be started. With `cmd` equal to NULL: nonzero, a shell being always present.

2.4 `popen()`, `pclose()`

Run a command and read its output, or write to its input.

Synopsis

```
#include <stdio.h>

FILE *popen(const char *cmd, const char *mode);
int pclose(FILE *stream);
```

Arguments

`cmd`

The command line, run by `/bin/sh` exactly as `system()` runs it.

`mode`

"`r`" to read the command's standard output, or "`w`" to write to its standard input.

`stream`

For `pclose`, the stream a previous `popen()` returned.

Description

Creates a pipe, spawns the shell with the pipe as its standard output (mode "`r`") or standard input (mode "`w`"), and returns a stream on this end of it. The child's descriptors are arranged with the file actions of §2.1, which is what lets `popen()` exist at all on a system without `fork()`.

`pclose()` closes the stream, waits for the child, and returns its status. A stream from `popen()` *must* be closed with `pclose()` and not `fclose()`: the latter would leave the child unreaped.

Returns

`popen()` returns a stream, or NULL with `errno` set. `pclose()` returns the child's termination status in the form `waitpid()` reports it, or `-1`.

2.5 `setsid()`

Start a new session.

Synopsis

```
#include <unistd.h>

pid_t setsid(void);
```

Description

Makes the calling process the leader of a new session and of a new process group within it, and detaches it from any controlling terminal it had. This is the first half of what a login does: `getty` calls it before handing over to `login`, so that the shell which eventually runs is the leader of a session of its own and a hangup on one terminal cannot reach another.

The second half — acquiring the *new* controlling terminal — is `tcsetctty()`, below.

Returns

The new session ID, which equals the caller's process ID, or `-1` with `errno` set.

Errors

EPERM

The caller is already a process-group leader, and so cannot start a session.

2.6 tcsetctty()

Acquire a controlling terminal by name.

Synopsis

```
#include <termios.h>

int tcsetctty(const char *path);
```

Arguments

path

The terminal to adopt, for example `/dev/con1` or `/dev/ser1`.

Description

Makes the named terminal the controlling terminal of the calling session, so that the terminal's interrupt and hangup characters reach this session's foreground process group.

QSOE note. This is a QSOE interface, not a POSIX one. On a Unix system a session leader acquires a controlling terminal as a side effect of opening a terminal that has none — an implicit rule that depends on an ordering the caller cannot see. QSOE makes the step explicit and gives it a name: nothing is acquired by accident, and a program that means to take a terminal says so. Callers coming from Unix should insert a `tcsetctty()` where they relied on the implicit behavior.

Returns

Zero on success, or `-1` with `errno` set.

Errors

ENOENT

No such terminal.

ENOTTY

The named path is not a terminal.

EPERM

The caller is not a session leader.

2.7 `execve()`, `execv()`

Replace the calling process image.

Synopsis

```
#include <unistd.h>

int execve(const char *path, char *const argv[], char *const envp[]);
int execv (const char *path, char *const argv[]);
```

Not implemented. QSOE has no `exec`. Both functions announce themselves on the console and fail with `ENOSYS`. They exist so that ported code links and so that a program which reaches one says so at the point of the call, rather than failing later somewhere unrelated.

Description

There is no way to replace a running process image in QSOE, and none is planned. `exec` is inseparable from `fork()` in practice — its purpose is to finish what a fork started — and the combination is what `posix_spawn()` replaces in one call.

Ported code that calls `execve()` after a `fork()` should be rewritten as a single `posix_spawn()`; the file actions of §2.1 carry whatever the code did between the two.

Returns

−1, with `errno` set to `ENOSYS`. There is no success case.

Chapter 3

Pipes and descriptors

A descriptor in QSOE is a connection identifier: the two namespaces are one, so a descriptor number *is* the thing `MsgSend()` addresses. That identity is why so little of this chapter is needed — most of what other systems provide as descriptor plumbing already exists as message plumbing — and it is also why the few functions here are worth reading carefully, because each is a place where the two views meet.

3.1 `pipe()`, `pipe2()`

Create a pipe.

Synopsis

```
#include <unistd.h>

int pipe(int fd[2]);
int pipe2(int fd[2], int flags);
```

Arguments

fd Filled in with the read end at `fd[0]` and the write end at `fd[1]`.

flags

For `pipe2`, `O_CLOEXEC` or zero.

Description

Asks the pipe manager for a new pipe and returns its two ends. Everything written to `fd[1]` is readable from `fd[0]`, in order; a reader of an empty pipe blocks until there is something to read or every writer has closed, and a writer blocks when the pipe is full.

QSOE note. `pipe2()` accepts only `O_CLOEXEC`. Any other flag is refused with `EINVAL` rather than accepted and ignored — `O_NONBLOCK` is a library-local placeholder on this system and would not be honored, and packet mode (`O_DIRECT`) is not implemented. Refusing is the honest answer: a program that asked for non-blocking behavior and silently did not get it fails much later and much less clearly.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EINVAL

A flag other than `O_CLOEXEC` was given to `pipe2()`.

EMFILE

No free descriptor pair.

ENOSPC

The pipe manager has no free pipe.

See also

`qsoe_capture()`, below; the file actions of §2.1, which are how a pipe end reaches a child.

3.2 `qsoe_capture()`

Create a pipe that never makes the writer wait.

Synopsis

```
#include <sys/qsoe.h>

int qsoe_capture(int fd[2]);
```

Description

Like `pipe()`, except that the write end never blocks: everything written is retained until it is read, however much that is and however late the reader arrives.

This exists because of a specific deadlock. Command substitution — `$(cmd)` in a shell — runs a command and collects its output. When the command is a shell *builtin*, one process is on both ends of the pipe, and an ordinary pipe with a fixed ring gives exactly the classic deadlock: the writer fills the ring and waits for a reader that is itself, and neither ever runs again. A capture removes the wait, and the substitution completes.

On a system with a writable filesystem this job is usually done with a temporary file. QSOE 0.2 has one writable filesystem and it is in RAM, so a capture is that temporary file without the filesystem in between.

QSOE note. This is a QSOE interface with no POSIX equivalent. The retained data is bounded by the pipe manager, so a capture is not an unlimited sink; a writer that exceeds the bound gets the ordinary full-pipe behavior back.

Returns

Zero on success, or `-1` with `errno` set. Errors are those of `pipe()`.

3.3 `dup()`

Duplicate a descriptor onto the lowest free number.

Synopsis

```
#include <unistd.h>

int dup(int fd);
```

Description

Returns a new descriptor naming the same open as `fd`, chosen as the lowest free number. It is `fcntl(fd, F_DUPFD, 0)`, and it shares its implementation with `dup2()` — the two differ only in whether the caller names the slot it wants.

QSOE note. Duplication is a real operation on both kernels rather than a bookkeeping change in the library: on QSOE/N it is a fresh connection plus an `_IO_DUP` message so the resource manager performs its own clone, and on QSOE/L it is a copy of the connection capability. The consequence worth knowing is that a server *sees* the duplication and can keep per-open state across it, which is what makes a duplicated descriptor behave like the original rather than like a second, unrelated open of the same path.

Returns

The new descriptor, or `-1` with `errno` set.

Errors

EBADF

`fd` is not an open descriptor.

EMFILE

No free descriptor.

3.4 `__qsoe__close__fds()`

Release every descriptor the process holds.

Synopsis

```
#include <sys/qsoe.h>

void __qsoe__close__fds(void);
```

Description

Closes each open descriptor, telling the owning resource manager as it goes. The library calls it on the way out of a process, so that a server does not go on holding per-open state for a client that has finished — which is what POSIX says happens at exit in any case.

QSOE note. This covers the *ordinary* end of a process. A crash or a kill leaves a server holding state anyway, because there is no client left to send anything; releasing that belongs to taskman, the one party that outlives the process and knows what it had open. The name is reserved (two leading underscores) because it is library machinery: applications have no reason to call it, and the correct way to release a descriptor early remains `close()`.

Returns

Nothing.

Chapter 4

Files, directories and attributes

QSOE 0.1 could read a file. QSOE 0.2 can create a directory, change a file's mode, owner and size, and make a temporary file with a name nobody else will guess. What made that possible is not any single function here but the arrival of a writable filesystem and, with it, servers that have something to change.

One consequence runs through the whole chapter and is stated once here rather than repeated in every entry: **every function below asks a resource manager to do the work**, and a resource manager that does not do writes refuses. The boot archive is read-only, so a `chmod()` of a file in it fails with `EROFS` — correctly, and by the same path that a real filesystem uses to succeed. Which filesystem a path lands on therefore decides whether these calls work, and `/tmp` (in RAM) and the mounted `/usr` behave differently on purpose.

4.1 `mkdir()`

Create a directory.

Synopsis

```
#include <sys/stat.h>

int mkdir(const char *path, mode_t mode);
```

Arguments

`path`

The directory to create.

`mode`

Permission bits for the new directory.

Description

Creates a directory, failing if the name already exists.

QSOE note. There is no `TM_REQ_MKDIR`. A directory is created by opening its name with `O_CREAT | O_EXCL | O_DIRECTORY` and closing the descriptor at once — the creation is entirely in the flags. This is deliberate: it gives a filesystem one creation path to implement rather than two, and a server that does not create directories fails the open, which is already the right answer for a read-only filesystem. Nothing about it is visible to a caller who uses `mkdir()`; it matters if you are writing a resource manager.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EEXIST

The name already exists.

ENOENT

A component of the path does not exist.

EROFS

The filesystem holding the path does not create names.

4.2 The `chmod` and `chown` families

Change a file's permission bits or ownership.

Synopsis

```
#include <sys/stat.h>
#include <unistd.h>

int chmod  (const char *path, mode_t mode);
int fchmod (int fd, mode_t mode);
int fchmodat(int dirfd, const char *path, mode_t mode, int flags);

int chown (const char *path, uid_t uid, gid_t gid);
int fchown(int fd, uid_t uid, gid_t gid);
int lchown(const char *path, uid_t uid, gid_t gid);
```

Arguments

`path, fd`

The file, named either by `path` or by an open descriptor.

`mode`

The new permission bits.

`uid, gid`

The new owner and group. Either may be $(uid_t) - 1$ or $(gid_t) - 1$ to leave that field alone.

`dirfd, flags`

For `fchmodat`, the directory a relative path is taken from and the resolution flags.

Description

All six are one request under the skin — `_IO_SETATTR`, carrying a mask of which fields are being set and the values for them. The mask is the point: a `chmod()` must not disturb the file's size and a `truncate()` must not disturb its mode, so “everything in the structure” would be the wrong contract. Each function sets one or two bits in the mask and leaves the rest alone.

The `path` forms and the `descriptor` forms differ only in where the request goes: a `path` form is sent to `taskman`, which resolves the name and forwards the identical frame to the owning server, while a `descriptor` form goes straight to the server the `descriptor` is already connected to.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EACCES

The caller is neither the file's owner nor root.

ENOENT

No such file.

ENAMETOOLONG

The path is longer than the request can carry.

EROFS

The file is on a filesystem that does not change attributes — the boot archive, for instance.

4.3 `truncate()`, `ftruncate()`

Set a file's length.

Synopsis

```
#include <unistd.h>

int truncate (const char *path, off_t length);
int ftruncate(int fd, off_t length);
```

Description

Sets the file's size to `length`, discarding anything beyond it or extending it with zeros. Like the mode and owner calls above, this is one field of an `_IO_SETATTR`, so a truncation leaves the mode and ownership untouched.

QSOE note. Partial shrink is not implemented in `fs-tmpfs`. Truncating a file to zero works; truncating it to a nonzero length smaller than its current size is refused with `EINVAL`. Extending works. This is a gap in the `tmpfs` server, not in the interface, and it is the one attribute operation that does not behave as POSIX describes.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EINVAL

Negative length, or a partial shrink on `fs-tmpfs`.

EROFS

The filesystem does not change file sizes.

4.4 mkstemp(), mkdtemp()

Create a uniquely named temporary file or directory.

Synopsis

```
#include <stdlib.h>

int  mkstemp(char *template);
char *mkdtemp(char *template);
```

Arguments

template

A writable string ending in exactly six **X** characters, which are replaced with the chosen name. It is modified in place.

Description

Chooses a name that does not yet exist, creates it, and reports it back through the template. `mkstemp()` creates a file and returns an open descriptor for it; `mkdtemp()` creates a directory, mode 0700, and returns the template pointer.

QSOE note. Both need a filesystem that creates names, which in QSOE 0.2 means `/tmp`. A template naming a directory on the boot archive or on the read-only `/usr` fails with `EROFS`.

Returns

`mkstemp()` returns an open descriptor, or `-1` with `errno` set. `mkdtemp()` returns `template`, or `NULL` with `errno` set.

Errors

EINVAL

The template does not end in six **X** characters.

EEXIST

No unused name was found after the permitted number of attempts.

EROFS

The containing filesystem does not create names.

4.5 rename()

Rename a file.

Synopsis

```
#include <stdio.h>

int rename(const char *oldpath, const char *newpath);
```

Not implemented. Not implemented. `rename()` announces itself on the console and fails with `EROFS` on every call.

Description

Renaming is the one filesystem write QSOE 0.2 does not perform. It is not a missing line of glue: a rename must be atomic with respect to both names, and on a filesystem that is still gaining its write path that is a promise better left unmade than made falsely. It is 0.3 work, alongside the rest of `qrvfs`'s writes.

Code that renames a file to publish it atomically — write to a temporary name, then rename over the target — has no equivalent here yet and must write to the target directly, accepting the window.

Returns

−1, with `errno` set to `EROFS`.

Chapter 5

Directory streams

QSOE 0.1 could walk a directory forwards, once, with `opendir()` and `readdir()`. This chapter is the rest of the interface: going back to the beginning, remembering a position and returning to it, reaching the descriptor underneath, and reading a whole directory into memory sorted.

The position functions rest on one fact worth stating: a QSOE directory stream is an ordinary open descriptor with a buffer in front of it, and a position in the directory is an offset on that descriptor. So `seekdir()` is an `lseek()` plus a discard of whatever the buffer had cached from the old position — and a directory that a server will not seek is one where these functions quietly do nothing rather than corrupting the walk.

5.1 `dirfd()`

Get the descriptor behind a directory stream.

Synopsis

```
#include <dirent.h>

int dirfd(DIR *d);
```

Description

Returns the file descriptor the stream is reading, so that calls taking a descriptor — `fstat()`, `fchmod()`, or the `dirfd` argument of `fchmodat()` — can be pointed at the directory itself.

The descriptor belongs to the stream. Closing it behind the stream's back, or reading from it directly, will confuse the buffered walk.

Returns

The descriptor, or `-1` with `errno` set to `EINVAL` if `d` is `NULL`.

5.2 Moving within a directory stream

Move about within a directory stream.

Synopsis

```
#include <dirent.h>

void rewinddir(DIR *d);
long telldir (DIR *d);
void seekdir (DIR *d, long off);
```

Arguments

d An open directory stream.

off

For `seekdir`, a position previously returned by `telldir()` on this same stream.

Description

`rewinddir()` returns the stream to the first entry. `telldir()` reports the current position, and `seekdir()` returns to one.

A position is meaningful only for the stream that produced it, and only while that stream is open. It is not a byte offset a caller may compute, and arithmetic on it is not defined.

QSOE note. `rewinddir()` and `seekdir()` return `void`, so a server that refuses to seek cannot be reported to the caller. When that happens the stream is left exactly where it was rather than being half-moved — the buffer is only discarded once the seek has succeeded, so a refused seek leaves a consistent stream rather than one that has forgotten its place. If you need to know whether a directory is seekable, seek the descriptor from `dirfd()` yourself and read the error.

Returns

`telldir()` returns the current position, or `-1` with `errno` set to `EINVAL` for a `NULL` stream. The other two return nothing.

5.3 scandir(), alphasort()

Read a whole directory into memory, filtered and sorted.

Synopsis

```
#include <dirent.h>

int scandir(const char *path, struct dirent ***namelist,
            int (*filter)(const struct dirent *),
            int (*compar)(const struct dirent **, const struct dirent **));

int alphasort(const struct dirent **a, const struct dirent **b);
```

Arguments

path

The directory to read.

namelist

Receives a pointer to an allocated array of pointers to allocated `struct dirents`.

filter

Called for each entry; a zero return drops it. `NULL` keeps everything.

compar

Passed to `qsort()` to order the result. `NULL` leaves the order the directory gave.

Description

Opens the directory, reads every entry, keeps those the filter accepts, sorts them, and hands back the array. `alphasort()` is the usual comparison, ordering by name.

The caller owns everything returned: each `struct dirent` and the array itself must be released with `free()`.

QSOE note. The whole directory is held in memory at once. On a system whose writable filesystem lives in RAM that is a real cost, and a large directory walked with `readdir()` costs a buffer rather than the whole listing. `scandir()` is the right tool when you need the entries sorted or counted; it is the wrong one for walking something large.

Returns

The number of entries in the array, or `-1` with `errno` set.

Errors**ENOENT**

No such directory.

ENOTDIR

The path is not a directory.

ENOMEM

The array or an entry could not be allocated.

Chapter 6

Formatted and line-oriented input

QSOE 0.1 could write formatted output and could not read it back. That asymmetry is more limiting than it sounds: a system whose configuration files, device descriptions and test fixtures are text needs to parse text, and without `scanf()` every program that read a number wrote its own parser. This chapter is the input half arriving.

It came with a floating-point scanner underneath it, which is why `strtod()` and its relatives appear in chapter 7: the same code serves `%f` in a format string and `strtod()` in a program, and the two arrived together.

6.1 The `scanf` family

Read formatted input.

Synopsis

```
#include <stdio.h>
#include <stdarg.h>

int scanf (const char *restrict fmt, ...);
int fscanf (FILE *restrict f, const char *restrict fmt, ...);
int sscanf (const char *restrict s, const char *restrict fmt, ...);

int vscanf (const char *restrict fmt, va_list ap);
int vfscanf(FILE *restrict f, const char *restrict fmt, va_list ap);
int vsscanf(const char *restrict s, const char *restrict fmt, va_list ap);
```

Arguments

f The stream to read, for the `fscanf` forms. The plain forms read `stdin`.

s The string to read, for the `sscanf` forms.

fmt
The conversion specification.

ap For the `v` forms, the argument list, already started.

Description

The six differ only in where the input comes from and how the arguments are passed; the conversion behavior is identical and is the standard one. The supported conversions include

the integer and floating forms (`%d %i %o %u %x %e %f %g %a`), strings and characters (`%s %c %[...]`), pointers (`%p`), the assignment-suppressing `*`, field widths, the length modifiers, and `%n`.

Three behaviors are worth confirming because programs depend on them and they are exercised by the system's own test suite: conversion stops at the first field that does not match; `%n` reports how far the scan got; and literal text in the format must match literal text in the input.

Returns

The number of items successfully assigned, which may be fewer than requested, or `EOF` if input ended before the first conversion.

Errors

EILSEQ

An input byte was not valid in the current encoding.

ENOMEM

An allocating conversion could not obtain memory.

Errors from the underlying stream are reported through the stream's error indicator, readable with `ferror()`.

See also

`strtod()` and the conversion functions of chapter 7, which share this family's scanner.

6.2 `getline()`, `getdelim()`

Read a line, or a delimited record, allocating as needed.

Synopsis

```
#include <stdio.h>

ssize_t getline (char **restrict lineptr, size_t *restrict n,
                FILE *restrict f);
ssize_t getdelim(char **restrict lineptr, size_t *restrict n,
                int delim, FILE *restrict f);
```

Arguments

`lineptr`

Address of a buffer pointer. Set it to `NULL` and `*n` to zero to have the function allocate; otherwise it grows the buffer with `realloc()` as needed.

n Address of the buffer's size, updated when the buffer grows.

`delim`

For `getdelim`, the byte that ends a record. `getline()` is `getdelim()` with a newline.

f The stream to read.

Description

Reads until the delimiter, end of file, or an error, storing the bytes in the buffer and terminating them with a NUL. The delimiter itself is kept in the buffer when one was found.

The buffer belongs to the caller and survives the call, so a loop reads every line without reallocating each time; it must be released with `free()` when the loop ends, including after a failing call.

QSOE note. These are the recommended way to read a line in QSOE. The obvious alternative — `fgets()` on a terminal — does not do what a Unix programmer expects here, because the console has *no line discipline*: there is no kernel-side editor accumulating a line, and `termios` is a library-local fake. A program that wants line editing on a terminal cooks its own input. For reading a file or a pipe, `getline()` is exactly what it appears to be.

Returns

The number of bytes placed in the buffer, not counting the terminating NUL, or `-1` at end of file or on error.

Errors

EINVAL

`lineptr` or `n` was `NULL`.

ENOMEM

The buffer could not be grown.

6.3 `rewind()`, `fgetpos()`, `fsetpos()`

Position a stream.

Synopsis

```
#include <stdio.h>

void rewind (FILE *f);
int fgetpos(FILE *restrict f, fpos_t *restrict pos);
int fsetpos(FILE *f, const fpos_t *pos);
```

Description

`rewind()` returns a stream to its beginning and clears its error indicator — it is `fseek(f, 0, SEEK_SET)` with the error flag cleared and no way to report failure.

`fgetpos()` and `fsetpos()` record a position and return to it. On QSOE an `fpos_t` is a byte offset, so they carry no more information than `ftell()` and `fseek()` do; they exist because the standard defines them and portable code uses them for streams whose position is not representable as a `long`.

Returns

`rewind()` returns nothing. The other two return zero on success, or `-1` with `errno` set.

Errors**EBADF**

The stream is not seekable.

EINVAL

The position argument was `NULL`, or names a place the stream cannot reach.

Chapter 7

Numeric conversion, mathematics and random numbers

Three small groups that arrived together and for one reason: QSOE 0.2 reads text that contains numbers. A configuration file with a frequency in it, a test that checks a computed value, a shell that evaluates arithmetic — each needs text to become a number and, often enough, a number to survive arithmetic that a long double can hold and a double cannot.

Note on long double

On RISC-V 64 a `long double` is `binary128` — quadruple precision — and the hardware does not implement it. Every long-double operation is therefore a call into the compiler's support library, which the C library links against. This is invisible in behavior and very visible in cost: a long-double multiply is orders of magnitude slower than a double one. Use `double` unless the extra range or precision is the point.

7.1 `strtod()`, `strtof()`, `strtold()`, `atof()`

Convert text to a floating-point number.

Synopsis

```
#include <stdlib.h>

double      strtod (const char *restrict s, char **restrict end);
float       strtof (const char *restrict s, char **restrict end);
long double strtold(const char *restrict s, char **restrict end);
double      atof   (const char *s);
```

Arguments

s The text to convert. Leading whitespace is skipped.

end

If not `NULL`, receives a pointer to the first character not consumed — which equals `s` when nothing converted.

Description

Converts an optionally signed decimal or hexadecimal floating constant, or one of the strings `inf`, `infinity` and `nan` in any letter case. Rounding is to nearest, as the current rounding mode directs.

`atof()` is `strtod(s, NULL)`: it cannot report where the number ended and cannot distinguish a failed conversion from a genuine zero. Prefer `strtod()` in anything that must validate its input.

Returns

The converted value. Zero if nothing converted. On overflow, $\pm\text{HUGE_VAL}$ (or the `float/long double` equivalent) with `errno` set to `ERANGE`; on underflow, a value no greater in magnitude than the smallest normalized number, likewise with `ERANGE`.

7.2 Floating-point building blocks

Floating-point building blocks.

Synopsis

```
#include <math.h>

double      scalbn   (double x, int n);
long double scalbnl  (long double x, int n);
long double fabsl    (long double x);
long double copysignl(long double x, long double y);
long double fmodl    (long double x, long double y);
```

Description

`scalbn()` and `scalbnl()` compute $x \times 2^n$ efficiently and without the intermediate overflow that writing the multiplication out would risk. `fabsl()` is magnitude; `copysignl()` takes the magnitude of `x` and the sign of `y`; `fmodl()` is the remainder of `x/y` with the sign of `x`.

These are here because the conversion functions above need them — a decimal to binary conversion is scaling and remainder work — and because they are the pieces on which the rest of a long-double math library is built as it arrives.

Returns

The computed value. `fmodl()` returns NaN with `errno` set to `EDOM` when `y` is zero.

7.3 llabs(), imaxabs()

Absolute value of a wide integer.

Synopsis

```
#include <stdlib.h>
#include <inttypes.h>

long long llabs  (long long n);
intmax_t  imaxabs(intmax_t n);
```

Description

The magnitude of `n`.

QSOE note. The most negative value of the type has no positive counterpart, and the result is then undefined — the same trap `abs()` carries, and worth restating because a 64-bit value makes the boundary easier to reach by accident, for instance when subtracting two timestamps in nanoseconds.

Returns

The absolute value.

7.4 `ffs()`, `ffsl()`, `ffsll()`

Find the first set bit.

Synopsis

```
#include <strings.h>

int ffs (int i);
int ffsl (long i);
int ffsll(long long i);
```

Description

Returns the position of the least significant set bit, counting from 1, or zero when the argument is zero. Note the one-based counting: it exists so that zero can mean “no bits set”, and it is a frequent source of off-by-one errors in code that mixes these with a shift.

Returns

The bit position, or zero.

7.5 `rand()`, `srand()`, `random()`, `srandom()`

Pseudo-random numbers.

Synopsis

```
#include <stdlib.h>

int rand (void);
void srand (unsigned seed);
long random (void);
void srandom(unsigned seed);
```

Description

`rand()` returns a value in $[0, \text{RAND_MAX}]$; `random()` returns one in $[0, 2^{31} - 1]$ from a generator with a longer period. Each is seeded by its own `srand/srandom`, and an unseeded generator behaves as if seeded with 1 — so a program that does not seed produces the same sequence on every run, which is useful for tests and wrong for anything else.

Not implemented. Neither is suitable for anything security-related. Both are deterministic sequences intended for simulation, sampling and tests.

QSOE note. QSOE has `/dev/urandom` — served by taskman from a SplitMix64 generator — and it is *also* not a cryptographic source. There is no CSPRNG in QSOE 0.2 and no `/dev/random`. A program that needs unpredictable bytes does not have a source for them yet, and should say so rather than reaching for one of these.

Returns

The next value in the sequence. The seeding functions return nothing.

Chapter 8

Strings, memory and multibyte characters

The string functions here are the ones a shell and a file manager turned out to need: case-insensitive comparison and search, substring search over arbitrary bytes, a bounded duplicate, and a thread-safe error message.

The multibyte functions are a different matter, and worth a paragraph of their own. QSOE has exactly one locale — `C.UTF-8` — and no locale subsystem: there is nothing to set, nothing to query, and no code path where the encoding is anything but UTF-8. That is a decision, not a gap. A locale subsystem is a large amount of machinery whose payoff is the ability to be wrong in more than one way, and a system whose text is UTF-8 everywhere can convert with three functions instead. `MB_CUR_MAX` is 4 and does not vary.

8.1 `strncasecmp()`

Compare two strings, up to a length, ignoring case.

Synopsis

```
#include <strings.h>

int strncasecmp(const char *a, const char *b, size_t n);
```

Description

Compares at most `n` bytes of the two strings, treating upper and lower case as equal, and reports their order.

QSOE note. Case folding is ASCII only. In a UTF-8 string the bytes of a non-ASCII character are compared unchanged, so this function answers correctly for ASCII text and *byte-identically* for the rest — which is the right behavior for the things it is used on here (option names, **yes/no** answers, file extensions) and the wrong tool for collating human-readable text, for which QSOE has no interface at all.

Returns

Negative, zero or positive as `a` sorts before, equal to, or after `b`.

8.2 strcasestr()

Find a substring, ignoring case.

Synopsis

```
#include <string.h>

char *strcasestr(const char *haystack, const char *needle);
```

Description

Locates the first occurrence of **needle** in **haystack**, comparing without regard to ASCII case. An empty needle matches at the start.

Returns

A pointer into **haystack**, or NULL if the needle is not present.

8.3 memmem()

Find a byte sequence within a byte sequence.

Synopsis

```
#include <string.h>

void *memmem(const void *haystack, size_t hlen,
             const void *needle, size_t nlen);
```

Description

Like **strstr()** but for arbitrary bytes: neither argument need be NUL-terminated and both may contain NUL. This is the function to reach for when searching a buffer that came off a wire or out of a file rather than a C string.

An empty needle matches at the start of the haystack.

Returns

A pointer into **haystack**, or NULL.

8.4 strndup()

Duplicate at most *n* bytes of a string.

Synopsis

```
#include <string.h>

char *strndup(const char *s, size_t n);
```

Description

Allocates and returns a copy of at most the first `n` bytes of `s`, always NUL-terminated — so the allocation is up to `n+1` bytes. The source need not be NUL-terminated within `n` bytes, which is what makes this the safe way to copy a field out of a buffer.

The result must be released with `free()`.

Returns

The new string, or NULL with `errno` set to `ENOMEM`.

8.5 `strerror_r()`

Render an error number into a caller-supplied buffer.

Synopsis

```
#include <string.h>

int strerror_r(int errnum, char *buf, size_t buflen);
```

Description

Writes the message for `errnum` into `buf`. Unlike `strerror()`, which may return a pointer to storage shared across the process, this form writes where the caller says and is therefore safe to use from more than one thread.

QSOE note. This is the POSIX form, returning an `int` — not the GNU form, which returns a `char *` and may leave the buffer untouched. Code written against the GNU form will compile here and then use an uninitialized pointer; it is a substitution worth checking for during a port.

Returns

Zero on success, or an error number.

Errors

`EINVAL`

`errnum` is not a known error number; the buffer still receives a generic message.

`ERANGE`

The buffer is too small for the message.

8.6 `dirname()`

Take the directory part of a path.

Synopsis

```
#include <libgen.h>

char *dirname(char *path);
```

Description

Returns the path up to, but not including, its final component. A path with no slash yields "."; the root yields "/".

QSOE note. `path` may be modified in place, and the result may point into it. Pass a copy if you still need the original — this is standard behavior and a standard source of surprise.

Returns

A pointer to the directory part.

8.7 `mbrtowc()`, `wcrtomb()`, `mbsinit()`

Convert between UTF-8 bytes and wide characters.

Synopsis

```
#include <wchar.h>

size_t mbrtowc(wchar_t *restrict pwc, const char *restrict s, size_t n,
               mbstate_t *restrict ps);
size_t wcrtomb(char *restrict s, wchar_t wc, mbstate_t *restrict ps);
int     mbsinit(const mbstate_t *ps);
```

Arguments

pwc

Where to store the decoded character, or `NULL` to decode without storing.

s The bytes to decode, or the buffer to encode into — at least `MB_CUR_MAX` bytes, which is 4.

n How many bytes at **s** may be examined.

wc The character to encode.

ps Conversion state, for a sequence that may be split across calls. `NULL` uses a per-function internal state, which is not thread-safe.

Description

`mbrtowc()` decodes one UTF-8 character; `wcrtomb()` encodes one. Both are restartable: a character split across two buffers is handled by carrying **ps** between the calls, which is what makes them the right tools for a stream. `mbsinit()` reports whether a state is at a character boundary.

QSOE note. The encoding is always UTF-8; there is no locale to change it. A program need not call `setlocale()` before using these, and calling it changes nothing.

Returns

`mbrtowc()` returns the number of bytes consumed; zero when the character was NUL; `(size_t)-2` when the bytes so far are a valid but incomplete prefix; `(size_t)-1` on an invalid sequence, with `errno` set to `EILSEQ`.

`wcrtomb()` returns the number of bytes written, or `(size_t)-1` with `EILSEQ` if `wc` is not a valid character.

`mbsinit()` returns nonzero if the state is initial.

Chapter 9

Time and clocks

QSOE 0.1 could read a clock. QSOE 0.2 can turn what it reads into a date, a date back into a time, and a difference between two of them into something a person can read — which is what a file manager showing modification times, a shell prompt with a clock in it, and a log with timestamps all needed.

One property of the whole chapter: **QSOE keeps universal time and has no time-zone database.** There is no `/usr/share/zoneinfo`, TZ is not consulted, and the local time is UTC. The functions below that are defined in terms of local time therefore behave as their UTC counterparts, and this is stated in each rather than left to be discovered.

9.1 mktime(), timegm()

Turn a broken-down time into a time value.

Synopsis

```
#include <time.h>

time_t mktime(struct tm *tm);
time_t timegm(struct tm *tm);
```

Arguments

tm The broken-down time. It is *normalized in place*: fields out of range are carried into the next field, and `tm_wday` and `tm_yday` are recomputed. That normalization is useful in its own right — adding 40 to `tm_mday` and calling `mktime()` is how you get the date 40 days hence.

Description

Converts a calendar date and time to seconds since the epoch.

QSOE note. `mktime()` interprets its argument as *universal* time, because QSOE's local time is UTC. It is therefore identical to `timegm()`. Both names exist so that portable code compiles unchanged, and code that carefully distinguished them elsewhere will find they agree here.

Returns

The time value, or `(time_t) - 1` if the date cannot be represented.

9.2 Broken-down time: `gmtime`, `asctime`, `ctime`

Turn a time value into a broken-down time or into text.

Synopsis

```
#include <time.h>

struct tm *gmtime    (const time_t *t);
char      *asctime    (const struct tm *tm);
char      *asctime_r  (const struct tm *restrict tm, char *restrict buf);
char      *ctime      (const time_t *t);
char      *ctime_r    (const time_t *restrict t, char *restrict buf);
```

Description

`gmtime()` breaks a time value into a `struct tm` in UTC. `asctime()` renders a broken-down time as the fixed 26-byte form "Wed Jun 30 21:49:08 1993\n". `ctime()` is `asctime(localtime(t))` — and since local time is UTC, it is `asctime(gmtime(t))`.

The `_r` forms write into a caller-supplied buffer, which must be at least 26 bytes.

QSOE note. `gmtime()`, `asctime()` and `ctime()` return pointers to storage shared across the process, so a second call overwrites the first and two threads calling them race. Use the `_r` forms, or `strftime()` for anything a person will read: the `asctime()` format is fixed, English, and not what you want in a user interface.

Returns

A pointer to the result, or NULL if the time cannot be represented.

9.3 `difftime()`

The difference between two times, in seconds.

Synopsis

```
#include <time.h>

double difftime(time_t a, time_t b);
```

Description

Returns $a - b$ as a `double`. On QSOE a `time_t` is a signed 64-bit count of seconds, so the subtraction could be written directly; the function exists because the C standard does not require that representation and portable code should not assume it.

Returns

The difference in seconds.

9.4 `clock__getres()`

Report a clock's resolution.

Synopsis

```
#include <time.h>

int clock_getres(clockid_t clk, struct timespec *ts);
```

Arguments

clk

The clock, for example `CLOCK_REALTIME` or `CLOCK_MONOTONIC`.

ts Receives the resolution — the smallest interval the clock distinguishes, not the smallest interval you can ask it to measure.

Description

Asks the kernel for the clock's tick period and reports it as a `timespec`. The value comes from `ClockPeriod()`, so it is the kernel's real answer for that clock on that board rather than a constant: a board whose timebase runs at a different frequency reports a different resolution, and a program that is timing something should read it rather than assume.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EINVAL

`ts` was `NULL`, or the clock id is not one this system knows.

9.5 usleep()

Suspend the caller for a number of microseconds.

Synopsis

```
#include <unistd.h>

int usleep(unsigned useconds);
```

Description

Sleeps for at least the requested interval. It is `nanosleep()` with the argument converted, and inherits its behavior exactly — including that a signal cuts the sleep short.

QSOE note. The interval is a *minimum*. The caller becomes runnable when it expires and then waits for the scheduler like anything else, so a higher-priority thread can extend the delay arbitrarily. A program with a deadline should express it as a deadline — with a timer — rather than as a sleep; see the *Programming Book* for the timer interfaces.

Returns

Zero on success, or `-1` with `errno` set to `EINTR` if a signal arrived first.

Chapter 10

Diagnostics

Two ways for a program to say something is wrong, for two different audiences.

The **err/warn** family writes to standard error for the person who typed the command. It is here because it is used by so much portable command-line code that its absence is felt on the first day of a port, and because the alternative — three lines of **fprintf()** and **strerror()** at every call site — is exactly the kind of repetition that ends up subtly inconsistent.

The system log is for whoever is holding the failure afterwards. QSOE 0.2 added a verbosity threshold to it, and where that threshold is applied turns out to matter a great deal on a message-passing system.

10.1 The **err** and **warn** family

Print a formatted diagnostic, and optionally exit.

Synopsis

```
#include <err.h>

_Noreturn void err  (int status, const char *fmt, ...);
_Noreturn void errx (int status, const char *fmt, ...);
void        warn  (const char *fmt, ...);
void        warnx(const char *fmt, ...);

_Noreturn void verr  (int status, const char *fmt, va_list ap);
_Noreturn void verrx (int status, const char *fmt, va_list ap);
void        vwarn  (const char *fmt, va_list ap);
void        vwarnx(const char *fmt, va_list ap);
```

Arguments

status

The exit status for the **err** forms, which do not return.

fmt

A **printf** format, or **NULL** to print only the program name and the error message.

ap For the **v** forms, an argument list already started.

Description

Each writes to standard error: the program name, then a colon, then the formatted message, then — for the forms *without* an **x** — a colon and the text of the current **errno**, and finally a newline.

The naming is worth learning once, because it is the whole family:

err vs. **warn**

err exits with **status**; **warn** returns.

the **x** suffix

Excludes the **errno** text. Use it when the failure was not a system call — a bad argument, a malformed file — so that a stale **errno** from something unrelated is not appended as if it explained anything.

the **v** prefix

Takes a **va_list**, for building your own wrappers.

QSOE note. The program name is the one **qsh** or **posix_spawn()** placed in **argv[0]**. A program spawned under a different **argv[0]** — which QSOE does deliberately, for instance to run **qedit** as **qview** — reports the name it was invoked under, not the file it was loaded from. That is the intended behavior and is what makes a multiple-personality program's diagnostics read correctly.

Returns

The **warn** forms return nothing. The **err** forms do not return.

10.2 slog_setlevel(), slog_getlevel()

Set or read the system-log verbosity threshold.

Synopsis

```
#include <sys/slog.h>

int slog_setlevel(int level);
int slog_getlevel(void);
```

Arguments

level

The lowest severity that will be recorded, from **_SLOG_ERROR** (least verbose) through **_SLOG_WARNING**, **_SLOG_INFO**, **_SLOG_DEBUG1** to **_SLOG_DEBUG2** (most). Out-of-range values are clamped rather than refused.

Description

Sets the threshold below which **slogf()** and its relatives discard a record. The default is **_SLOG_INFO**, so the two debug levels cost nothing until something asks for them.

QSOE note. The filtering happens *at the call site* — before the clock is read, before the message is formatted, and before anything is written to `/dev/slog`. A suppressed `slogf()` therefore costs a load, a compare and a return.

This is deliberately not where QNX filters. Its `slogger` daemon drops records the client has already formatted and copied, which on QSOE would mean paying a system call and a full IPC round trip for every line that is thrown away — on a message-passing system that is not a small difference, and on a driver's hot path it is the difference between a log level you can leave enabled and one you cannot.

The scale is 1:1 with `libtaskman`'s `tm_log`, so a level chosen for a driver means the same thing in `taskman`.

Returns

`slog_setlevel()` returns the *previous* threshold, so a caller that raises it temporarily can put it back. `slog_getlevel()` returns the current one. Neither fails.

Chapter 11

System identity and configuration

Which machine is this, what is it called, and who is the caller? QSOE 0.2 answers all three, and it answers them the same way on both kernels because all three answers come from taskman.

The hostname is the thread running through the chapter. It is not a string the library keeps: it lives in taskman, one copy for the machine, published at `/sys/hostname` and adopted by the network station as its call sign. A system whose machines can see each other's files needs exactly one answer to “which machine is this”, and putting that answer anywhere else would make two.

11.1 `uname()`

Describe the system.

Synopsis

```
#include <sys/utsname.h>

int uname(struct utsname *u);
```

Arguments

u Receives the description. The structure holds `sysname`, `nodename`, `release`, `version` and `machine`, each a NUL-terminated string.

Description

Fills in the five fields:

sysname

QSOE/N or QSOE/L, naming the kernel this system runs on. This is the field to read when a program must know which variant it is on — and it should be reading it, not guessing from anything else.

nodename

The hostname.

release

The QSOE version.

version

The build date.

machine
riscv64.

QSOE note. The first four are read from `/sys` at each call — they are taskman’s answers, not constants compiled into the library, which is what lets one `libc.so` give the right answer on either kernel. A field whose `/sys` entry cannot be read comes back as an empty string rather than as stale or invented text.

Returns

Zero on success, or `-1` with `errno` set to `EINVAL` if `u` is `NULL`.

11.2 gethostname(), sethostname()

Read or set this machine’s name.

Synopsis

```
#include <unistd.h>

int gethostname(char *name, size_t len);
int sethostname(const char *name, size_t len);
```

Arguments

name

The buffer to fill, or the name to set.

len

The buffer’s size, or the length of the name being set (not counting a terminating NUL).

Description

`gethostname()` asks taskman for the machine’s name and copies it in, always NUL-terminating. `sethostname()` changes it.

QSOE note. There is no `gethostid()` and no numeric host identity. A machine has a name; the network layer has a *station id*, assigned by hand in `/etc/qsp.conf` and printed on the front panel, and the two are deliberately different things — see chapter 14. Code that used a host id as a unique machine number wants the station id.

Returns

Zero on success, or `-1` with `errno` set.

Errors

ENAMETOOLONG

The buffer is too small for the name, or the name being set is longer than `HOST_NAME_MAX`.

EPERM

`sethostname()` was called by a process that is not root.

11.3 qsoe__become__user()

Authenticate as a user and adopt that user’s credentials.

Synopsis

```
#include <sys/qsoe.h>

int qsoe_become_user(const char *name, const char *password);
```

Arguments

name

The account to become.

password

Its password, in the clear. It is sent to taskman as its own message fragment and is never copied into an intermediate buffer by the library.

Description

Checks the password against the credential files and, if it matches, changes the calling process’s user and group identity to that account’s. This is what **login** calls; it is the only way to change identity in QSOE.

QSOE note. There is deliberately no companion “check this password” call. Split into two operations — verify, then become — the pair is a password oracle with no rate limit behind it: any process could test an account’s password and read the answer off which call failed. As a single operation, a wrong password and a right one differ only in whether the caller is now somebody else.

For the same reason there is one error code for both “no such user” and “wrong password”. Distinguishing them would tell an attacker which half to keep guessing.

Returns

Zero on success, or `-1` with **errno** set.

Errors

EACCES

No such user, or the password does not match. The two are not distinguished.

EINVAL

A `NULL` or empty name, or a name or password longer than the request can carry.

11.4 qsoe__sysfiles__reload()

Ask taskman to re-read the credential files.

Synopsis

```
#include <sys/qsoe.h>

int qsoe_sysfiles_reload(void);
```

Description

taskman caches `/etc/passwd`, `/etc/group` and `/etc/shadow`, because it must answer credential questions without going to a filesystem every time. This call tells it the files have changed.

It is needed at exactly one moment in an ordinary boot: the credential files live on `/usr`, which is mounted after taskman has started, so the cache taskman built at boot is the boot archive's copy until something says otherwise. The init script makes this call after the mount.

QSOE note. The caller must be a third party — specifically, it must not be the filesystem server that holds the files. taskman reads them as a client of that server, and its dispatch is single-threaded, so a reload request arriving *from* that server would park taskman on a server that is itself parked on this reply: a deadlock, with both sides waiting correctly. The init script and an operator command are the intended callers.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EPERM

The caller is not root.

EIO

A file could not be read; the previous cache is kept rather than being replaced by a partial one.

Chapter 12

Device control

This chapter has one function in it, and it is the substitution a porter meets first.

`ioctl()` does not exist in QSOE and will not. Its defects are structural rather than incidental: a single signed integer carries the subsystem, the direction of travel and the command, packed by macros that different systems pack differently, so a request code is not portable, is not checkable, and — because the integer is signed — overflows into negative values for perfectly ordinary codes. That last one is not hypothetical: QSOE hit it during 0.2, where a request code above `INT_MAX` was truncated by the parameter's type, a `switch` matched it and an equality test did not, and a terminal on the video console silently lost its window-size query.

`devcontrol()` carries the same three things as three unsigned values.

12.1 `devcontrol()`

Send a typed control request to a resource manager.

Synopsis

```
#include <sys/qsoe.h>
#include <qsoe/ctlcodes.h>

ssize_t devcontrol(int fd, unsigned subsys_dir, unsigned cmd,
                  void *data, size_t in, size_t cap);
```

Arguments

fd An open descriptor on the object to control.

subsys_dir

The subsystem being addressed in the high 16 bits, and the direction of travel in the low 16. Build it with `QSOE_CTL_SD(subsys, dir)`; never by hand.

The subsystems are catalogued in `<qsoe/ctlcodes.h>` — `QSOE_SUBSYS_TERM` for terminals, `QSOE_SUBSYS_VIDEO`, `QSOE_SUBSYS_BLOCK`, `QSOE_SUBSYS_NET`, and so on, with `QSOE_SUBSYS_USER` reserved for application-defined ones.

The direction is `QSOE_CTL_DIR_IN`, `QSOE_CTL_DIR_OUT`, both (a get-and-set in one call), or `QSOE_CTL_DIR_NONE` for a command that carries no data at all.

cmd

The subsystem's own command ordinal. Shared subsystems define theirs in `<qsoe/ctlcodes.h>`; a server with a vocabulary of its own defines them in its own header.

data

The buffer: **in** bytes travel to the server, and up to **cap** bytes come back into the same place.

in, cap

How many bytes go out, and how many may come back.

Description

Reaches the `control()` method of whichever resource manager owns **fd** — the typed escape hatch the resource-manager framework provides for everything that is not a read, a write or a stat.

The direction is the part that earns its place. Because the caller *declares* which way bytes travel, the framework checks that declaration against **in** and **cap** before the server's method is entered at all. A request that says it carries input and passes a zero count is refused at the boundary rather than inside a driver that then reads an uninitialized buffer. With `ioctl()` the direction is encoded in the request number by a macro, which means it is a claim nobody verifies.

Two subsystems — `QSOE_SUBSYS_CORE` and `QSOE_SUBSYS_FSYS` — may be addressed to a provider of *any* kind, and a refusal from one of those is an answer rather than a missing implementation. That is how **df** finds the filesystems: it asks every mount point a filesystem question and reads the refusals.

QSOE note. Porting from `ioctl()`. Each request becomes a (**subsys**, **dir**, **cmd**) triple. A request with no data becomes `QSOE_CTL_DIR_NONE` with **in** and **cap** both zero; an `_IOR` becomes `DIR_OUT` with **cap** set; an `_IOW` becomes `DIR_IN` with **in** set; an `_IOWR` becomes both. The command ordinals are yours to choose — they only have to be unique within your subsystem, not globally, which is the other thing `ioctl()` never managed.

Returns

The number of bytes returned in **data**, which is zero for a command that returns none, or `-1` with **errno** set.

Errors**EBADF**

fd is not open.

EINVAL

The declared direction does not match the counts given.

ENOSYS

The server has no `control()` method at all.

ENOTTY

The server has one, but does not answer for this subsystem — for instance a terminal question asked of a block device.

EOVERFLOW

cap is too small for the answer.

See also

`<qsoe/ctlcodes.h>` for the subsystem catalogue; the *Design & Architecture* guide's resource-manager chapter for the server-side `control()` method.

Chapter 13

Shared memory and physical allocation

Message passing is how QSOE processes talk, and for almost everything that is the right cost. For a few things it is not: a text console repainted at 30 frames a second, a network ring, a framebuffer. These want one process to *look at* another's memory rather than to be sent a copy of it.

The interface for that is a *region*: memory one process owns and offers, and another maps. What makes it safe is that neither party ever names an address. The owner publishes a range and receives an opaque id; the recipient presents the id and receives a mapping. taskman brokers the pair, so a process cannot map memory it was not offered by naming a lucky address, and a grant is exactly as wide as what was published.

How the grant is satisfied differs completely between the kernels — Skimmer maps the owner's frames by physical address, while QSOE/L copies the frame capabilities into the recipient's CSpace — and the caller cannot tell. What comes back is a mapped range either way, which is the whole point of the seam.

13.1 qsoe_region_publish()

Offer a range of your memory to another process.

Synopsis

```
#include <sys/qsoe.h>
#include <qsoe/tm_msgs.h>

int qsoe_region_publish(void *va, unsigned long bytes, pid_t peer,
                        unsigned flags, unsigned *id_out);
```

Arguments

va, bytes

The range being offered. It must be memory the caller actually owns as one run — see the note below.

peer

The process allowed to map it.

flags

QSOE_REGION_F_SHARED for a live view — writes by either side are seen by the other — or QSOE_REGION_F_SNAPSHOT for a copy taken at publish time.

id_out

Receives the region id to hand to the peer.

Description

Registers the range with taskman and returns an id naming it. The id is what travels to the other process, by whatever means the two already use to talk; it carries no address and is useless to anyone the region was not published to.

QSOE note. The range must be one run of frames the caller owns. A region assembled from separately obtained allocations is refused, and the refusal names the address — “published via ... that is not one run it owns”. In practice this means publishing memory you obtained in a single `qsoe_mmap()` or `qsoe_alloc_phys()` call, not a range you computed across two. The check exists because a partially valid region would otherwise be granted and then fault inside the recipient, a long way from the mistake.

Returns

Zero on success, or `-1` with `errno` set.

Errors**EINVAL**

The range is not one run the caller owns, or the flags are contradictory.

ENOSPC

taskman’s region table is full, or the run needs more frames than a region may hold.

ESRCH

No such peer process.

13.2 qsoe__map__region()

Map a region that was published to you.

Synopsis

```
#include <sys/qsoe.h>

int qsoe_map_region(unsigned id, unsigned flags,
                   void **va_out, unsigned long *bytes_out);
```

Arguments

id The region id, as received from its owner.

flags

The properties required of the region — pass the QSOE_REGION_F_* value you expect, or zero to accept whatever it was published as.

va_out, bytes_out

Receive the address in *this* process and the length.

Description

Asks taskman for the region and maps it. The address returned is this process's own; it will not equal the publisher's, and nothing may be exchanged between the two processes that assumes it does — an offset within the region travels, a pointer does not.

The mapping covers exactly what was published, including a sub-page offset: if the owner published a range starting part-way into a page, the returned address points at that byte and the length is the published length, not the page-rounded one.

Returns

Zero on success, or `-1` with `errno` set.

Errors

EACCES

The region was not published to this process.

EINVAL

The required flags do not match how the region was published.

ENOENT

No such region, or it has been revoked.

ENOMEM

No address space to map it into.

13.3 qsoe_region_revoke()

Withdraw a region you published.

Synopsis

```
#include <sys/qsoe.h>

int qsoe_region_revoke(unsigned id);
```

Description

Withdraws the offer, so that no further `qsoe_map_region()` succeeds for that id.

QSOE note. Revoking does not tear down a mapping already made. A peer that mapped the region keeps its view until it unmaps or exits, so revocation closes the door rather than emptying the room. A publisher that must know when its memory is no longer visible has to arrange that with the peer directly — there is no notification.

Returns

Zero on success, or `-1` with `errno` set.

Errors

ENOENT

No such region.

EPERM

The caller does not own it.

13.4 qsoe_alloc_phys_flags()

Allocate memory and learn its physical address, with attributes.

Synopsis

```
#include <sys/qsoe.h>

void *qsoe_alloc_phys_flags(unsigned long length, int prot,
                           unsigned long flags, uint64_t *pa_out);
```

Arguments

length, prot

As for `qsoe_mmap()`.

flags

`QSOE_ALLOC_UNCACHED` for memory the CPU will not cache, or zero.

pa_out

Receives the physical address backing the allocation.

Description

The same as `qsoe_alloc_phys()`, which is this function with `flags` zero, plus allocation attributes. It exists for drivers: hardware that is handed an address needs the physical one, and hardware that is not cache-coherent needs memory the CPU is not caching behind its back.

QSOE note. `QSOE_ALLOC_UNCACHED` is not a substitute for cache maintenance on a platform that needs it, and it is much slower for anything the CPU touches often. A driver on a non-coherent SoC generally wants ordinary cached memory plus explicit clean and invalidate operations around the DMA; uncached memory is for the small structures where the maintenance would cost more than the caching saves — a descriptor ring’s head and tail, a mailbox.

Returns

The virtual address, or `QSOE_MAP_FAILED` with `errno` set.

Errors

EINVAL

A zero length, or an unknown flag.

ENOMEM

No contiguous physical memory of that size is available.

EPERM

The caller is not permitted physical allocations.

13.5 qsoe_fdt_region()

Map the firmware device tree.

Synopsis

```
#include <sys/qsoe.h>

int qsoe_fdt_region(void **out_va, unsigned long *out_size);
```

Description

Maps the flattened device tree the firmware handed the kernel, read-only, and reports where and how large it is. This is how a driver learns what its hardware actually is — register windows, interrupt numbers, clock frequencies, PHY addresses — rather than carrying a table of constants per board.

The mapping is read-only and shared: every process that asks sees the same bytes, and the device tree is not writable by anyone.

Returns

Zero on success, or `-1` with `errno` set.

Errors

ENOENT

The firmware supplied no device tree.

ENOMEM

No address space to map it into.

Chapter 14

Node descriptors and the net

QSOE 0.2 put machines on a net together. Most of what that added is not in the C library at all: a station appears as a directory under `/net`, and a program reaches another machine's files by opening a path. `open()`, `read()` and `close()` do the work, and the fact that a wire was involved never reaches the program — which is the whole design.

What the library did gain is the small piece that cannot be a path: turning a station's call sign into the *node descriptor* that QNX-style calls take, and back again. This chapter is those two functions and the macros that go with them. The protocol itself, the roster file, and the `/net` namespace are the subject of the *Networking Guide* (QSOE-NET).

What a node descriptor is

A node descriptor — an `nd` — is a 32-bit value naming a station and, optionally, which link to reach it by. The station id is in the low 16 bits and a path selector in the high 16:

```
#define QSP_ND_STATION(nd)      /* the station id */
#define QSP_ND_PATH(nd)        /* which link to it */
#define QSP_ND_MAKE(st, path)  /* build one */
#define QSP_ND_CMP(a, b)       /* compare identity, ignoring the link */
```

The path selector exists because a station can be reachable by more than one route — the VisionFive 2 carries two Ethernet MACs — and an application with a real-time stream may want it pinned to one of them. It is deliberately excluded from `QSP_ND_CMP()`: two descriptors naming the same machine by different links are the *same node*, exactly as QNX defines it.

QSOE note. A node descriptor is obtained, never constructed. Get one from `qsp_strtond()` or from resolving a path under `/net`. The encoding above is documented as a debugging convenience, not as a contract: leaving it unfixed is what allows high station ids to become per-process handles later without breaking the ABI, and code that builds an `nd` by shifting bits is code that will need editing when that happens.

Two station numbers are reserved: `QSP_STATION_LOCAL` (zero) means this machine, and `QSP_STATION_BROADCAST` means all of them.

14.1 `qsp_strtond()`

Resolve a call sign to a node descriptor.

Synopsis

```
#include <qsoe/qsp.h>

uint32_t qsp_strtond(const char *callsign);
```

Arguments

`callsign`

A station's name, as it appears under `/net` — short, printable, and without slashes or dots, because it becomes a directory name.

Description

Asks the local network manager to look the name up in the station roster and returns the corresponding node descriptor.

QSOE note. The lookup goes to the *manager*, not to a copy of the roster inside the library. That is not an implementation detail: a name that resolved differently depending on which process asked would be worse than a name that did not resolve at all, and a cached roster in every process is exactly how the two come to disagree. There is one roster on a station, and this reads it.

The three outcomes are distinguished, and the distinction is useful:

- **No manager running** means this machine is not on a net at all, which is `ENETDOWN`.
- **A manager that has never heard of the station** means the roster does not list it — a configuration mistake, reported as `EHOSTUNREACH`.
- **A listed station that is not currently answering resolves successfully.** Being unreachable is a property of the moment, not of the name; the caller finds out when it tries to connect, and gets a different error then.

Returns

The node descriptor, or zero with `errno` set. Zero is also `QSP_STATION_LOCAL`, so a caller that must distinguish “the local station” from a failure should clear `errno` first and test it after.

Errors

`ENETDOWN`

No network manager is running on this machine.

`EHOSTUNREACH`

The roster does not name that station.

`EINVAL`

A NULL or empty call sign, or one longer than `QSP_CALLSIGN_MAX`.

14.2 `qsp_ndtostr()`

Render a node descriptor as a call sign.

Synopsis

```
#include <qsoe/qsp.h>

int qsp_ndtostr(uint32_t nd, char *buf, unsigned cap);
```

Arguments

nd The node descriptor to name.

buf, cap

Where to write the name, and how much room there is. A buffer of `QSP_CALLSIGN_MAX + 1` bytes is always enough.

Description

The inverse of `qsp_strtond()`: asks the manager which station the descriptor names and writes its call sign. This is what turns an `nd` that arrived in a message into something a person can read in a log line.

Returns

The number of bytes written, not counting the terminating NUL, or `-1` with `errno` set.

Errors

ENETDOWN

No network manager is running.

EHOSTUNREACH

The descriptor names no station in the roster.

ERANGE

The buffer is too small for the call sign.

See also

The *Networking Guide* (QSOE-NET) for the `/net` namespace, the roster file, and how a path on another station is opened.